

NAME: Riya Dinesh Paturde

Batch:B2

Roll NO: B-24

Aim: Data ingestion and cleaning : Acquire data and prepare it for analysis

Theory:

1]what is data ingestion and why is it important in data science? which python library is commonly used for data ingestion and cleaning?

Ans: Data ingestion is the process of transporting data from one or more sources to a target site (like a database, data warehouse, or a notebook) for storage and analysis.

Sources can include:

Local files (CSV, Excel, JSON) SQL/NoSQL databases

Real-time streaming data (APIs, IoT sensors)

Cloud storage (S3, Google Cloud Storage)

Data ingestion is an important step because it prepares data for further processes like cleaning, processing, and analysis.

if the data is not properly ingested, it may lead to incorrect results. this is known as Garbage In, Garbage Out.

Data ingestion is important due to the following reasons:

Accessibility: It brings fragmented data into a centralized environment where it can actually be used.

Data Velocity: It allows data scientists to handle data at scale, whether it's a one-time upload or a continuous stream.

Foundation for Cleaning: You can't fix what you haven't captured yet. pandas is widely used for data ingestion and cleaning in data science. It uses read_ functions (like read_csv) for ingestion and provides powerful methods (like dropna() or fillna()) for cleaning.

2]why is data cleaning necessary before performing analysis or machine learning?

Ans: Data cleaning is necessary because raw data is almost always "dirty," containing errors that can lead to misleading conclusions or complete model failure.

Here is why it's a critical step before any analysis:

1 Accuracy and Reliability If your data contains incorrect entries (e.g., a person's age listed as 200), your averages, correlations, and predictions will be skewed. Cleaning ensures the insights you extract are based on reality, not noise.

2 The "Garbage In, Garbage Out" Principle In machine learning, a model is only as good as the data used to train it. If you feed a model biased, duplicate, or incorrect data, it will learn those exact errors and produce "garbage" predictions.

3. Compatibility with Algorithms Most machine learning algorithms cannot handle:

Missing Values: Many models will simply crash if they encounter a NaN or Null value.

Non-Numeric Data: Models usually require numbers. Cleaning involves converting text (like "Male/Female") into formats the math can understand.

4. Reducing Bias Duplicate entries or over-represented "junk" data can cause a model to become unfairly biased toward certain outcomes. Cleaning helps balance the dataset for a more objective analysis.

5. Efficiency Working with a smaller, streamlined, and "clean" dataset speeds up processing time and reduces the computational power required to run complex models.

3] why is data cleaning necessary before performing analysis or machine learning?

Ans: Data cleaning is a critical step because raw data is almost always "noisy" or "dirty."

If you feed poor-quality data into an analysis or a machine learning model, the results will be unreliable—a concept known as Garbage In, Garbage Out (GIGO).

Data cleaning is necessary before analysis or machine learning for these three primary reasons:

Model Reliability (GIGO): The principle of "Garbage In, Garbage Out" applies. If the input data is incorrect, duplicate, or noisy, the resulting insights or model predictions will be fundamentally flawed.

Algorithmic Requirements: Most machine learning models (like those in scikit-learn) cannot handle missing values (NaN) or text data. Cleaning ensures data is complete and numerical.

Statistical Accuracy: Outliers and errors can heavily skew averages and correlations. Cleaning ensures your analysis reflects real-world patterns rather than data entry errors.

To load a CSV file into a Pandas DataFrame, you use the `read_csv()` function. Basic syntax: `import pandas as pd`
`df = pd.read_csv('your_file_name.csv')`

4] What is difference between `head()` and `tail()` functions in pandas?

Ans: head() function Purpose:

Returns the first rows of the DataFrame. Default: If you don't specify a number, it shows the first 5 rows.

Use Case: Best for checking if your headers (column names) loaded correctly and seeing the initial data entries.

tail() function purpose:

Returns the last rows of the DataFrame. Default: If you don't specify a number, it shows the last 5 rows.

Use Case: Best for checking the end of the file to see if there are any "junk" summary rows, totals, or trailing empty rows that need to be cleaned.

5]how can you identify missing values in a dataset?

Ans: in pandas, missing values in a data set can be identified using built in function like isnull() and isna().

this function help to detect empty or null values (such as nan). You can identify them using these three primary methods:

df.isnull().sum(): This is the most common method. It returns the total count of missing values for every column in the DataFrame.

df.info(): This provides a summary of the dataset. By comparing the "Non-Null Count" to the total number of entries, you can quickly see which columns have missing data.

df.isnull().any(): This returns a boolean (True/False) for each column, indicating if it contains any missing values at all.

```
import pandas as pd
import numpy as np
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Alice', np.nan],
    'Age': [25, 30, 200, 25, 22],
    'City': ['New York', 'Los Angeles', np.nan, 'New York', 'Chicago'],
    'Sales': ['100', '200', '150', '100', '300']
}
df = pd.DataFrame(data)
print("--- Original Data ---")
print(df)
df = df.drop_duplicates()
df['City'] = df['City'].fillna('Unknown')
df = df.dropna(subset=['Name'])
df['Sales'] = df['Sales'].astype(int)
median_age = df[df['Age'] < 100]['Age'].median()
df.loc[df['Age'] > 100, 'Age'] = median_age
```

```
print("\n--- Cleaned Data ---")
print(df)
print("\n--- Data Info ---")
df.info()
```

```
--- Original Data ---
```

	Name	Age	City	Sales
0	Alice	25	New York	100
1	Bob	30	Los Angeles	200
2	Charlie	200	NaN	150
3	Alice	25	New York	100
4	NaN	22	Chicago	300

```
--- Cleaned Data ---
```

	Name	Age	City	Sales
0	Alice	25.0	New York	100
1	Bob	30.0	Los Angeles	200
2	Charlie	27.5	Unknown	150

```
--- Data Info ---
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 3 entries, 0 to 2
```

```
Data columns (total 4 columns):
```

```
#   Column  Non-Null Count  Dtype
```

```
---  ---
0   Name    3 non-null    object
1   Age     3 non-null    float64
2   City    3 non-null    object
3   Sales   3 non-null    int64
```

```
dtypes: float64(1), int64(1), object(2)
```

```
memory usage: 120.0+ bytes
```

```
/tmp/ipykernel_27237/3633790829.py:17: FutureWarning: Setting an item of incompatible dtype
df.loc[df['Age'] > 100, 'Age'] = median_age
```

conclusion:

We have demonstrated the process of loading raw data, identifying and handling issues such as duplicate entries, missing values, and outliers, and converting data types. The dataset has been transformed into a clean and structured format, making it suitable and reliable for subsequent exploratory data analysis and machine learning model development.

